

61. Rotate List

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 10, 2026

Problem Summary

Given the head of a linked list and integer k , rotate the list to the right by k places and return the new head.

Approach

Find the tail and length in one pass, make the list circular by connecting tail to head, then find the new tail at position $\text{count} - k$ from the start, break the circle there and return the new head.

Algorithm

1. Handle edge cases: $\text{head} == \text{null} \parallel \text{head.next} == \text{null} \parallel k == 0$.
2. Traverse to tail while counting length (count), curr ends at tail.
3. $k = k \% \text{count}$ — reduce redundant full rotations. If $k == 0$ return head.
4. $\text{idx} = \text{count} - k$ — position of new tail from start (1-based).
5. Walk newTail from head until $i == \text{idx}$.
6. $\text{curr.next} = \text{head}$ — make circular.
7. $\text{head} = \text{newTail.next}$ — new head.
8. $\text{newTail.next} = \text{null}$ — break circle.
9. Return head.

Key Insights

- Rotating right by k is equivalent to taking the last k nodes and moving them to the front.
- $k \% \text{count}$ eliminates full rotations — rotating a list of length 5 by 5 is a no-op.
- Making the list **circular first**, then breaking at the right point is cleaner than trying to rewire multiple pointers at once.
- New tail is at position $\text{count} - k$ from start (1-based) — the node just before the new head.

Observations

- curr naturally lands at tail after the first while loop — reused for making the circle.
- $\text{idx} = \text{count} - k$ with 1-based indexing means the while loop runs $\text{idx} - 1$ times (stops when $i == \text{idx}$).
- Edge case $k \% \text{count} == 0$ must be caught **after** computing $k \% \text{count}$, not before.
- First while stops at $\text{curr.next} != \text{null}$ (tail), not $\text{curr} != \text{null}$ — key difference from standard traversal.

Points to Remember

1. **Rotate = circular + break pattern**: connect tail to head $\rightarrow$ find new tail $\rightarrow$ break. Cleaner than multi-pointer rewiring.
2. $k \% \text{count}$ is mandatory — always reduce k before using it; k can be larger than list length.
3. $\text{idx} = \text{count} - k$ converts "k from end" to "position from start" — same idea as LC 19 ($\text{size} - n$).
4. Always handle $k \% \text{count} == 0$ after reduction — not just $k == 0$ upfront.
5. Order of the last three lines matters: make circular $\rightarrow$ set new head $\rightarrow$ break circle. Swapping them causes NPE or wrong result.

Time Complexity & Space Complexity

$O(n)$ — one pass to find tail + length, one pass to find new tail.

$O(1)$ — only pointers.

Linked List

DescriptionEditorialSolutionsSubmissions

61. Rotate List

Solved

MediumTopicsCompanies

Given the head of a linked list, rotate the list to the right by k places.

Example 1:

1 → 2 → 3 → 4 → 5

rotate 15 → 1 → 2 → 3 → 4

rotate 24 → 5 → 1 → 2 → 3

Input: head = [1,2,3,4,5], k = 2

Output: [4,5,1,2,3]

Example 2:

0 → 1 → 2

rotate 12 → 0 → 1

rotate 21 → 2 → 0

rotate 30 → 1 → 2

rotate 42 → 0 → 1

11.5K27091 Online

Accepted

232 / 232 testcases passed

Sa... submitted at Jul 10, 2026 16:41

AnalysisSolution

Runtime

0 msBeats 100.00%

Memory

44.41 MBBeats 26.17%

100%

50%

0%

1ms2ms3ms4ms

Code | Java

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val =
9  * 
```

View more

Write your notes here

Code

JavaAuto

```
11 class Solution {
12     public ListNode rotateRight(ListNode head, int k) {
13         if(head == null || head.next == null || k == 0) return head;
14
15         int count = 1;
16         ListNode curr = head;
17         while(curr.next != null){
18             count++;
19             curr = curr.next;
20         }
21
22         k = k % count;
23         if(k == 0) return head;
24
25         int idx = count - k;
26         int i = 1;
27         ListNode newTail = head;
28         while(i != idx){
29             newTail = newTail.next;
30             i++;
31         }
32
33         curr.next = head;
34         head = newTail.next;
35         newTail.next = null;
36
37         return head;
38     }
39 }
```

SavedLn 35, Col 37

Testcase | Test Result

AcceptedRuntime: 0 ms

Case 1Case 2